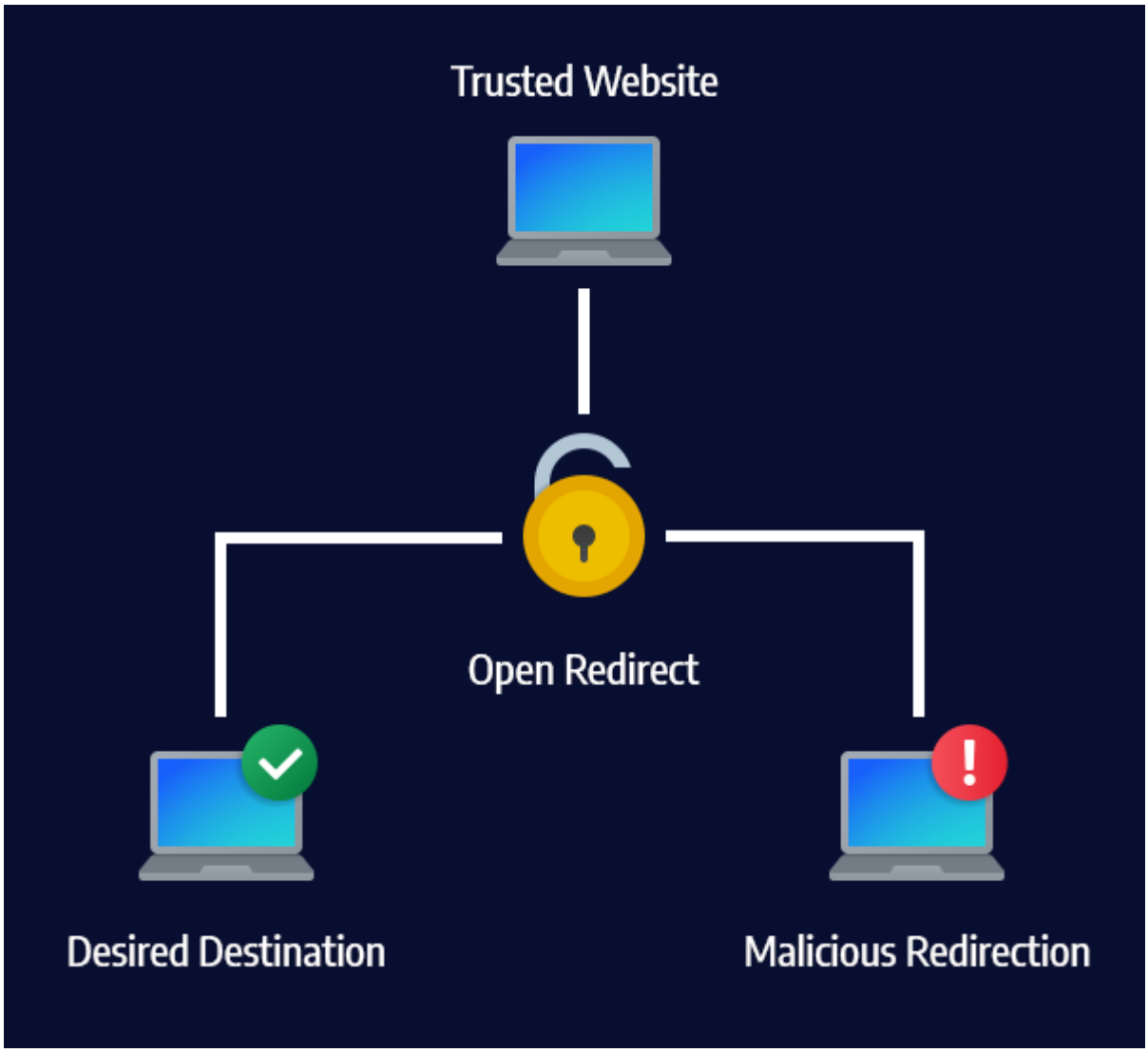
Open Redirection



Open redirection vulnerabilities arise when an application incorporates user-controllable data into the target of a redirection in an unsafe way. An attacker can construct a URL within the application that causes a redirection to an arbitrary external domain. This behaviour can be leveraged to facilitate phishing attacks against application users. The ability to use an authentic application URL, targeting the correct domain and with a valid SSL certificate (if SSL is used), lends credibility to the phishing attack because many users, even if they verify these features, will not notice the subsequent redirection to a different domain.

Here are some examples of safe redirects and unsafe redirects:

- If the legitimate website redirects the client to a fixed URL, it is a safe redirect.
- If the legitimate website safely constructs the redirect URL based on parameters provided by the user, it is a safe redirect.
- If the legitimate website constructs the redirect URL based on parameters provided by the user but does not sufficiently validate/filter input, it is an unsafe redirect (the attacker may manipulate the input).
- If the legitimate website allows the user to specify the destination redirect URL, it is an unsafe redirect (open redirect).

For example, if your domain is *example.com*, the attacker may create the following URL:

```
• https://example.com/redirect.php?url=http://attacker.com
```

Open redirection attacks are especially dangerous because an attacker knows that we're trying to log into a specific website, which makes us vulnerable to a phishing attack. For example, an attacker could send malicious emails to website users in an attempt to capture their passwords.

Different kinds of Open Redirects

➢ Header based

Header-based is a location header sent from the server. The benefit with this, from an attacker's perspective, is that the redirect always works even if Javascript is not interpreted. A server-side function that gets a URL as input will follow the redirect and end up somewhere else.

➢ Javascript based

When the redirect instead happens in Javascript it only works in scenarios where Javascript is executed. It might not work for server-side functions, but it will work in the victim's web browser.
If the redirect happens in Javascript it might also be possible to cause a redirect to javascript: something(), which would be an XSS in itself.

Impact

If you have an open redirection vulnerability, it makes many other attacks possible:

- **Phishing:** The most obvious way to use an open redirect is to steer the victim away from the original site to a site that looks the same, steal user credentials, and then return to the vulnerable website as if nothing happened.

- **Cross-site Scripting (XSS):** If the redirect allows the use of *data:* or *javascript:* protocols and the client supports such protocols in redirects, it makes it possible for the attacker to perform an XSS attack.

- **Server-Side Request Forgery (SSRF):** Open redirects may be used to evade SSRF filters.

- **Content-Security-Policy bypassing**: If you use CSP to protect against XSS and one of the whitelisted domains has an open redirect, this vulnerability may be used to bypass CSP.

- **CRLF Injection:** If the redirection parameter allows line breaks, the attacker may try to perform response header splitting.

How To Prevent Open Redirection

If possible, applications should avoid incorporating user-controllable data into redirection targets. In many cases, this behaviour can be avoided in two ways:

- Remove the redirection function from the application, and replace links to it with direct links to the relevant target URLs.

- Maintain a server-side list of all URLs that are permitted for redirection. Instead of passing the target URL as a parameter to the redirector, pass an index into this list.

If it is considered unavoidable for the redirection function to receive user-controllable input and incorporate this into the redirection target, one of the following measures should be used to minimize the risk of redirection attacks:

- The application should use relative URLs in all of its redirects, and the redirection function should strictly validate that the URL received is relative.

- The application should use URLs relative to the web root for all of its redirects, and the redirection function should validate that the URL received starts with a slash character. It should then prepend http://yourdomainname.com to the URL before issuing the redirect.

- The application should use absolute URLs for all of its redirects, and the redirection function should verify that the user-supplied URL begins with http://yourdomainname.com/ before issuing the redirect.

Conclusion

We now know how harmful an open redirect vulnerability can be and why it is so important to prevent it. The easiest and the most effective way to prevent it is by not letting your user have control of where the page redirects them to. It is only necessary to provide users with the authority to enter short keywords and map them to the target URL or simply allow them to confirm before being redirected to the destination.

References

- **wiki.owasp.org**
- **portswigger.net**
- **acunetix.com**
- **techtarget.com**